

TUGAS UAS WEEK 15

ROBOTIKA

Nama : Bayu Aditya Ramadhani Putra

Kelas : TK-45-01

NIM :1103210219

Chapter 1 : Introduction to ROS

ROS (Robot Operating System) adalah kerangka kerja (framework) perangkat lunak open-source yang dirancang untuk pengembangan aplikasi robotika. Meskipun disebut sebagai "sistem operasi," ROS sebenarnya bukan sistem operasi seperti Linux atau Windows. ROS adalah middleware yang menyediakan berbagai alat, pustaka, dan konvensi untuk membangun aplikasi robot secara efisien. ROS adalah framework fleksibel yang menyediakan berbagai alat dan pustaka untuk pengembangan perangkat lunak robotika. Keunggulan utama ROS meliputi:

- **Kemampuan tinggi:** Mendukung fungsi seperti SLAM (Simultaneous Localization and Mapping), navigasi, dan perencanaan gerak robot.
- **Beragam alat:** Alat visualisasi seperti RViz, simulator seperti Gazebo, dan alat debugging seperti rqt_gui.
- **Dukungan sensor dan aktuator:** Integrasi mudah untuk perangkat seperti LIDAR, kamera kedalaman, dan aktuator.
- **Modularitas:** Memisahkan proses dalam node independen, yang meningkatkan toleransi kesalahan dan kemudahan debugging.

Tujuan dan Fungsi Utama ROS:

1. Middleware untuk Robotika: ROS bertindak sebagai penghubung antara berbagai komponen robot, seperti sensor, aktuator, dan algoritma. ROS memungkinkan komunikasi antar komponen melalui mekanisme seperti pesan (messages), topik (topics), dan layanan (services).
2. Standarisasi dalam Robotika: ROS menyediakan standar untuk pengembangan perangkat lunak robotika, termasuk format pesan, API komunikasi, dan alat simulasi.
3. Komponen Modular: Aplikasi di ROS terdiri dari node independen. Setiap node menangani fungsi spesifik, seperti membaca sensor, mengontrol motor, atau menjalankan algoritma navigasi.

Memulai ROS

1. Instalasi: Disarankan menggunakan Ubuntu 20.04 dengan ROS Noetic.
Disarankan install ROS nya mengikuti arahan yang ada di website officialnya.Yaitu:
 - Setup your sources.list
Setup your computer to accept software from packages.ros.org. (sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu \$(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list')
 - Set up your keys
sudo apt install curl # if you haven't already installed curl (curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -)
 - Installation
First, make sure your Debian package index is up-to-date: (sudo apt update)
 - Desktop-Full Install: (Recommended) : Everything in Desktop plus 2D/3D simulators and 2D/3D perception packages (sudo apt install ros-noetic-desktop-full)
 - Desktop Install: Everything in ROS-Base plus tools like rqt and rviz (sudo apt install ros-noetic-desktop)
 - ROS-Base: (Bare Bones) ROS packaging, build, and communication libraries. No GUI tools. (sudo apt install ros-noetic-ros-base)
2. Menjalankan ROS Master:
 - Jalankan perintah roscore untuk memulai ROS Master, Parameter Server, dan node logging.
 - ROS Master bertindak sebagai pengatur komunikasi antara node.
 - kurang lebih hasil runningnya akan seperti ini:

```
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubunturos:43169/
ros_comm version 1.17.0

SUMMARY
=====
PARAMETERS
* /roscdistro: noetic
* /rosversion: 1.17.0

NODES

auto-starting new master
process[master]: started with pid [10659]
ROS_MASTER_URI=http://ubunturos:11311/

setting /run_id to 946d1e49-ca7c-11ef-a59e-6b6384fec540
process[roscout-1]: started with pid [10669]
started core service [/roscout]
```

Kesimpulan

Chapter ini memberikan landasan konsep dasar ROS, termasuk bagaimana ROS memanfaatkan komunikasi terdistribusi dan mengelola perangkat lunak robotika secara modular. Langkah berikutnya adalah memahami pengelolaan paket ROS dan membuat contoh implementasi praktis.

Chapter 2 : Getting started with ROS

Chapter 2 membahas langkah-langkah awal dalam pemrograman ROS, dimulai dengan membuat ROS packages sebagai unit dasar perangkat lunak ROS. Sebuah package berisi file penting seperti kode sumber, file konfigurasi, dan informasi dependensi. Untuk memulai, pengguna perlu membuat workspace menggunakan perintah `catkin_make` dan menyusun package dengan `catkin_create_pkg`. Workspace ini memungkinkan pengembangan dan pengelolaan banyak paket sekaligus.

1. Membuat ROS Package

- **ROS Package:** Unit dasar dalam ROS yang berisi file seperti kode sumber, konfigurasi, dan dependensi.
- **Langkah Membuat Package:**
 1. Buat workspace:

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
catkin_init_workspace
```

2. Buat package baru menggunakan:

```
catkin_create_pkg <package_name> <dependencies>
```

3. Build workspace:

```
cd ~/catkin_ws
catkin_make
```

2. Membuat ROS Nodes

- Node: Unit eksekusi independen dalam ROS, yang biasanya menangani satu fungsi spesifik.
- ROS mendukung dua bahasa utama:
 - Python (rospy): Mudah untuk implementasi cepat.
 - C++ (roscpp): Kinerja lebih tinggi untuk aplikasi kompleks.
- Contoh node python

```
import rospy
from std_msgs.msg import String

def simple_node():
    rospy.init_node('example_node', anonymous=True)
    rospy.loginfo("Node is running")
    rospy.spin()

if __name__ == '__main__':
    simple_node()
```

3. Bekerja dengan ROS Topics

- Topics: Kanal komunikasi unidirectional antara node (*publish/subscribe*).
- Mekanisme Komunikasi:
 - Publisher: Node yang mengirim data.
 - Subscriber: Node yang menerima data.
- Contoh Python Publisher:

```
pub = rospy.Publisher('chatter', String, queue_size=10)
pub.publish("Hello ROS!")
```

- Contoh Python Subscriber:

```
def callback(data):
    rospy.loginfo(f"I heard: {data.data}")
rospy.Subscriber('chatter', String, callback)
```

4. Hasil Run

```
vboxuser@ubuntu:~$ roslaunch my_pubsub publisher.py
[INFO] [1735982243.136406]: Hello ROS! 1735982243.1363542
[INFO] [1735982244.138443]: Hello ROS! 1735982244.1383348
[INFO] [1735982245.137409]: Hello ROS! 1735982245.1373014
[INFO] [1735982246.137186]: Hello ROS! 1735982246.137074
[INFO] [1735982247.139019]: Hello ROS! 1735982247.138904
[INFO] [1735982248.140481]: Hello ROS! 1735982248.1403735
[INFO] [1735982249.137469]: Hello ROS! 1735982249.137347
[INFO] [1735982250.137256]: Hello ROS! 1735982250.137149
[INFO] [1735982251.137353]: Hello ROS! 1735982251.137253
[INFO] [1735982252.138409]: Hello ROS! 1735982252.1383116
[INFO] [1735982253.139681]: Hello ROS! 1735982253.1395774
[INFO] [1735982254.136806]: Hello ROS! 1735982254.1366997
[INFO] [1735982255.138103]: Hello ROS! 1735982255.1380029
[INFO] [1735982256.136713]: Hello ROS! 1735982256.1366134
[INFO] [1735982257.137426]: Hello ROS! 1735982257.1372864
[INFO] [1735982258.137210]: Hello ROS! 1735982258.1371057
[INFO] [1735982259.142630]: Hello ROS! 1735982259.142497

vboxuser@ubuntu:~$ roslaunch my_pubsub subscriber.py
[INFO] [1735982285.138458]: Received message: Hello ROS! 1735982285.1368957
[INFO] [1735982286.141796]: Received message: Hello ROS! 1735982286.1395278
[INFO] [1735982287.142202]: Received message: Hello ROS! 1735982287.1403902
[INFO] [1735982288.138851]: Received message: Hello ROS! 1735982288.1368508
[INFO] [1735982289.139466]: Received message: Hello ROS! 1735982289.1376863
[INFO] [1735982290.139753]: Received message: Hello ROS! 1735982290.1380365
[INFO] [1735982291.140008]: Received message: Hello ROS! 1735982291.1379917
[INFO] [1735982292.138283]: Received message: Hello ROS! 1735982292.1367009
[INFO] [1735982293.138604]: Received message: Hello ROS! 1735982293.1369917
[INFO] [1735982294.139443]: Received message: Hello ROS! 1735982294.137849
[INFO] [1735982295.139760]: Received message: Hello ROS! 1735982295.1382036
[INFO] [1735982296.141071]: Received message: Hello ROS! 1735982296.139201
[INFO] [1735982297.139384]: Received message: Hello ROS! 1735982297.1377094
[INFO] [1735982298.139449]: Received message: Hello ROS! 1735982298.1378777
[INFO] [1735982299.139327]: Received message: Hello ROS! 1735982299.137986
[INFO] [173598300.139219]: Received message: Hello ROS! 173598300.1378136
[INFO] [173598301.139362]: Received message: Hello ROS! 173598301.1382332
[INFO] [173598302.138362]: Received message: Hello ROS! 173598302.1368964
[INFO] [173598303.141464]: Received message: Hello ROS! 173598303.140072
```

Penjelasan Hubungan Publisher-Subscriber

- Node publisher.py mempublikasikan pesan "Hello ROS!" secara terus-menerus pada sebuah topik (kemungkinan bernama chatter atau topik serupa).
- Node subscriber.py berlangganan ke topik yang sama dan menerima pesan tersebut.
- Timestamp pada kedua output membantu memverifikasi bahwa pesan dikirim dan diterima hampir secara real-time.

Chapter 3 : turtlesim in ROS

Turtlesim adalah sebuah package dalam ROS (Robot Operating System) yang digunakan untuk memberikan pengenalan dasar tentang bagaimana bekerja dengan ROS dan bagaimana perintah-perintah dalam ROS dapat mengontrol sebuah robot atau objek di dunia virtual. Turtlesim biasanya digunakan untuk tujuan pembelajaran, terutama untuk memahami konsep-konsep dasar dalam ROS seperti node, publisher, subscriber, dan topik.

Turtlesim Noetic adalah implementasi dari Turtlesim yang berjalan di ROS Noetic Ninjemys, yang merupakan distribusi ROS yang ditujukan untuk versi Ubuntu 20.04 (Focal Fossa). ROS Noetic adalah distribusi ROS terakhir yang didukung secara penuh dengan pembaruan dan perbaikan.

1. Instalasi ROS Noetic dan Turtlesim

Untuk menggunakan Turtlesim di ROS Noetic, Anda perlu terlebih dahulu menginstal ROS Noetic dan package Turtlesim. Berikut adalah cara instalasi ROS Noetic:

- **Langkah 1:** Install ROS Noetic

```
sudo apt update
sudo apt install ros-noetic-desktop-full
```

- **Langkah 2:** Setelah ROS Noetic terinstal, Anda bisa menginstal Turtlesim dengan perintah:

```
sudo apt install ros-noetic-turtlesim
```

2. Mengenal Struktur Turtlesim

Pada Turtlesim, kita akan melihat sebuah objek yang bernama "turtle" yang bergerak di dalam sebuah dunia virtual. Dengan menggunakan perintah yang tersedia, kita dapat mengontrol posisi dan orientasi turtle tersebut.

Ada beberapa elemen dasar dalam Turtlesim yang perlu dipahami:

- **Node:** Turtlesim berjalan sebagai sebuah node ROS yang dapat menerima input dan menghasilkan output dalam bentuk topik.
- **Topik (Topic):** Turtlesim menggunakan beberapa topik untuk berkomunikasi, seperti topik untuk pergerakan, posisi, dan orientasi turtle.
- **Publisher:** Turtlesim mengirimkan data tentang posisi turtle dan status lainnya melalui publisher ke topik yang relevan.
- **Subscriber:** Pengguna dapat berlangganan ke topik tertentu untuk mendapatkan informasi dari Turtlesim.

3. Menjalankan Turtlesim

Untuk menjalankan Turtlesim di ROS Noetic, pertama buka terminal dan gunakan perintah berikut:

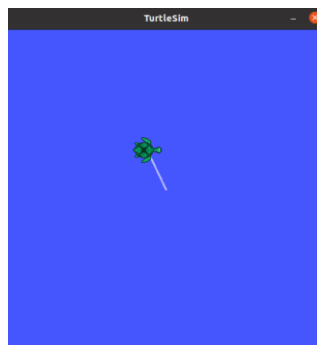
```
roscore
```

Ini akan memulai ROS Master. Setelah itu, di terminal lain, jalankan Turtlesim dengan perintah:

```
roslaunch turtlesim turtlesim_node
```

Ini akan membuka jendela GUI yang menampilkan dunia Turtlesim dan turtle yang bisa digerakkan.

```
vboxuser@ubuntu:~$ roslaunch turtlesim turtlesim_node
[ INFO] [1735982917.351098594]: Starting turtlesim with node name /turtlesim
[ INFO] [1735982917.356610446]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
```



4. Menggerakkan Turtle

Untuk menggerakkan turtle, kita bisa menggunakan perintah `turtle_teleop_key` yang memungkinkan mengontrol turtle menggunakan keyboard. Jalankan perintah ini di terminal lain setelah Turtlesim berjalan:

```
vboxuser@ubuntu:~$ roslaunch turtlesim turtle
turtlesim_node    turtle_teleop_key
vboxuser@ubuntu:~$ roslaunch turtlesim turtle_teleop_key
Reading from keyboard
-----
Use arrow keys to move the turtle. 'q' to quit.
```

Dengan perintah ini, kita dapat menggunakan tombol keyboard untuk menggerakkan turtle ke atas, bawah, kiri, kanan, dan memutar orientasi turtle. Turtlesim Noetic merupakan alat yang sangat baik untuk memulai belajar ROS, baik untuk pemula yang baru mengenal robotika atau bagi mereka yang ingin memahami alur kerja ROS secara lebih mendalam.

Chapter 4 : Gazebo in ROS

Gazebo adalah simulator robot yang memungkinkan pengguna untuk membuat dunia 3D virtual untuk menguji dan mensimulasikan berbagai jenis robot. Gazebo menyediakan model fisika yang realistis, termasuk dinamika, deteksi kolisi, dan sensasi sensor, yang memungkinkan pengujian di lingkungan virtual yang menyerupai dunia nyata. Fitur utama Gazebo meliputi:

- **Simulasi fisika:** Gazebo menggunakan engine fisika (misalnya ODE, Bullet, DART) untuk mensimulasikan gerakan objek, gravitasi, dan interaksi antar objek.
- **Sensor Simulasi:** Gazebo memungkinkan simulasi berbagai sensor robot, seperti LIDAR, kamera, IMU (Inertial Measurement Unit), dan sensor sentuh.
- **Model Robot:** Gazebo mendukung berbagai model robot, termasuk robot wheeled, drone, humanoid, dll. Model ini bisa dibuat dengan menggunakan URDF (Unified Robot Description Format) yang kompatibel dengan ROS.
- **Visualisasi:** Gazebo menyediakan visualisasi dunia simulasi secara 3D yang sangat mendetail, memungkinkan kita untuk melihat bagaimana robot berinteraksi dengan lingkungannya.

Untuk mengintegrasikan Gazebo dengan ROS, digunakan **gazebo_ros package**, yang menghubungkan ROS dengan Gazebo, menyediakan alat untuk mengendalikan simulasi dan berkomunikasi antara ROS dan Gazebo. Komponen utama gazebo_ros:

- **gazebo_ros_pkgs:** Paket ROS yang menyediakan integrasi antara Gazebo dan ROS. Paket ini mencakup berbagai plugin dan API untuk mengontrol dunia simulasi.
- **Gazebo ROS Plugins:** Plugin ini digunakan untuk mengendalikan elemen-elemen dalam Gazebo, seperti mengontrol robot, sensor, dan lainnya. Plugin ini memungkinkan untuk mengontrol elemen-elemen dalam dunia simulasi menggunakan topik, layanan, dan parameter ROS.
- **Model URDF dan SDF:** Model robot dapat didefinisikan menggunakan URDF (Robot Description) di ROS dan SDF (Simulation Description Format) di Gazebo. ROS menggunakan URDF untuk mendefinisikan struktur robot, sedangkan Gazebo menggunakan SDF untuk mendefinisikan dunia dan model robot.

Berikut adalah langkah-langkah sederhana untuk menggunakan Gazebo dengan ROS:

1. **Menjalankan Gazebo dengan ROS:** Untuk menjalankan simulasi dengan ROS di Gazebo, kita menggunakan roslaunch dengan file launch yang mengonfigurasi dunia dan robot.

```
roslaunch gazebo_ros empty_world.launch
```

Perintah ini membuka Gazebo dengan dunia kosong.

2. **Menambahkan Model ke Gazebo:** Anda dapat menambahkan model robot ke dunia simulasi menggunakan file URDF dan plugin Gazebo. Misalnya, untuk menambahkan robot wheeled:

```
roslaunch gazebo_ros spawn_model.launch model:=robot.urdf
```

3. **Kontrol Robot di Gazebo:** ROS dapat mengontrol robot di Gazebo dengan mengirimkan perintah ke robot melalui topik atau layanan ROS. Misalnya, Anda bisa mengendalikan gerakan robot menggunakan perintah ROS atau menggunakan node kontrol yang ditulis dalam Python atau C++.
4. **Simulasi Sensor di Gazebo:** Anda dapat menambahkan sensor ke robot (seperti kamera atau LIDAR) di Gazebo. Data sensor ini akan dipublikasikan ke topik ROS, yang kemudian dapat diproses dengan algoritma ROS.

Hasil Run

